SCRIPT REFERENCE / UNITY

# HS_Laser

## User Guide

Setup, materials and shader integration.
Shot control, collision and optional damage.




**LineRenderer + ParticleMesh**

English edition / Clickable contents & PDF bookmarks

CONTENTS

# Find what you need

A practical guide to configuring, rendering and controlling HS_Laser. Inspector field names are kept in English so you can follow along directly in Unity.


| | |
|---|---|
| Quick start | 3 |
| Shot timing & size | 4 |
| Materials: LineRenderer | 5 |
| Materials: ParticleMesh | 6 |
| Hit & Flash effects | 7 |
| Collision & damage | 8 |
| Connect an existing enemy | 9 |
| Event-based enemy integration | 10 |
| Origin damage & chains | 11 |
| Flicker & target impacts | 12 |
| Control from code | 13 |
| Troubleshooting | 14 |


**TWO RENDERING PATHS**

LineRenderer uses a two-point line with length-aware texture tiling. ParticleMesh builds a segmented beam and passes geometry and dissolve parameters to a compatible shader.

No enemy, spawner or demo controller script is required to display the laser. Damage integration and pooled target impact effects are optional.

01 / USER GUIDE

# Quick start

Start from a prepared laser object or prefab. HS_Laser controls the effect; it does not create its materials, mesh or textures.

## 1. Add the component

Place HS_Laser on the same GameObject as the main LineRenderer or ParticleSystem.

## 2. Select the renderer

Choose Laser Type. Auto prefers a LineRenderer; if none exists, it uses the ParticleSystem.

## 3. Assign a material

Use a material compatible with the selected mode. ParticleMesh also needs a configured mesh particle renderer.

## 4. Set the direction

The beam starts at the component's transform position and travels along its local +Z axis.

## 5. Configure the shot

Set Max Length and the Growth, Hold and Fade durations. Enable Play On Enable for automatic firing.

## 6. Add secondary effects

Assign child objects to Hit Effect and Flash Effect if needed. Each uses its own particle settings and materials.

> **SUGGESTED FIRST SHOT**
>
> Max Length: 12 · Growth: 0.2 s · Hold: 1 s · Fade: 0.2 s. Keep parent transforms at unit scale for predictable sizing.

Set the renderer type and effect references before the first activation: these components are cached during initialization. A blank Particle System with a generic material is not a ready-to-use volumetric laser.

02 / USER GUIDE

# Shot timing & size

| | |
|---|---|
| Growth Duration | Extends from zero toward Max Length. A blocking collider can shorten the visible beam. |
| Hold Duration | Keeps the beam active after growth finishes. |
| Fade Duration | Runs shutdown. Damage has already stopped. |
| Continuous | Ignores Hold Duration and stays active until DisablePrepare() is called. |
| Play On Enable | Automatically fires when enabled. |
| Deactivate When Finished | Disables the GameObject after completion; does not destroy it. |

**FADE IS DIFFERENT IN EACH MODE**

ParticleMesh receives a _Dissolve value from 0 to 1 during Fade. The LineRenderer switches off immediately when Fade begins; Fade Duration does not animate line opacity.

| | |
|---|---|
| Max Length | Maximum main-beam length in world units. |
| Laser Size | Scales the object and affects beam width. Does not change Max Length. |
| Laser Scale | Additional segment and width scale for ParticleMesh only. |
| Laser Color | Sets the main ParticleMesh particle color. Does not recolor a LineRenderer or child effects. |

Growth, Hold and Fade durations are captured by Fire(). Changes to these values apply to the next shot. Timing uses Time.deltaTime and follows Time.timeScale.

HS_Laser follows its current transform direction. Aiming, orbiting around a character and choosing the main shot direction belong to an external controller.

03 / USER GUIDE

# Materials: LineRenderer

The material defines the visual style: textures, color, emission, UV movement, masks and noise. The script updates only the properties needed to render and control the beam.

**PER-RENDERER VALUES**

HS_Laser uses MaterialPropertyBlock. It supplies values to each renderer without editing the shared material or creating a material copy through Renderer.material.

The script sets two world-space line positions and changes widthMultiplier while retaining the original width curve. LineRenderer UVs and Texture Mode still affect how textures appear.

| | |
|---|---|
| Main Texture Length | Sets X tiling in _MainTex_ST: MainTextureLength × CurrentLength / LaserSize. |
| Randomize Main Offset | Chooses a random U offset for the main texture. |
| Tile Noise | Enables script-controlled noise tiling and offset. |
| Noise Texture Name | Shader texture reference name; default _Noise. The script appends _ST. |
| Noise Texture Length | Sets noise X tiling using the same length/size relationship. |
| Randomize Noise Offset | Chooses a random U offset for noise. |
| Randomize Every Frame | Updates enabled random offsets every frame. |

Main Texture Length is a tiling multiplier, not the length of one texture in metres. Use an appropriate texture Wrap Mode for repeated patterns.

Y tiling and V offset are read from the material at initialization. U offset is zero when randomization is disabled. Without Randomize Every Frame, offsets are chosen at initialization, not on every shot.

The shader must actually use _MainTex_ST and the selected noise *_ST in its UV calculation. Texture names alone do not guarantee support. Set line color through the material and LineRenderer color gradient, if the shader uses vertex color.

04 / USER GUIDE

# Materials: ParticleMesh

HS_Laser places mesh particles along the beam and provides the shader with the current geometry and shutdown state. A compatible volumetric shader uses these inputs to shape the beam and limit it to its endpoints.

| | |
|---|---|
| _StartPoint | Beam origin in world space. |
| _EndPoint | Beam endpoint in world space. |
| _Distance | Current actual beam length. |
| _Scale | Laser Scale × Laser Size × current flicker multiplier, with a minimum clamp on the base scale. |
| _Dissolve | 0 during emission; rises to 1 during Fade. |

Laser Color becomes the main particles' color. The final result also depends on material color, textures and whether the shader reads vertex color.

## Particle setup

The script sets the main Particle System to Local Simulation Space, disables its normal emission and supplies particles itself. Its regular emission rate does not determine beam length.

## Texture movement and appearance

The script's Main Texture Length, Tile Noise and Randomize controls are used only in LineRenderer mode. Configure ParticleMesh texture tiling and motion in the material.

The supplied volumetric shader exposes _MainTexTilingXYNoiseTilingZW for main/noise tiling and _SpeedMainTexUVNoiseZW for their UV speeds. Other available material controls remain part of the visual setup.

**WHAT THE SCRIPT DOES NOT CHANGE**

HS_Laser does not assign textures, _Emission, _Color, UV speed or material switches. Runtime _Scale and _Dissolve overrides take precedence over the material values on the beam renderer.

Hit and Flash use their own renderers and materials. Main-beam color, tiling and property overrides are not automatically copied to these child effects.

05 / USER GUIDE

# Hit & Flash effects

Flash Effect is the visual at the beam origin. Hit Effect is the visual on a blocking surface. Assign child objects containing Particle Systems.

| | |
|---|---|
| Hit Offset | Offsets the surface effect along the hit normal. |
| Use Laser Rotation | Uses the laser rotation for Hit. Otherwise, it faces along the surface normal. |
| No blocking hit | Stops Hit particle emission. |
| Pierced enemy | Does not trigger the main surface Hit on that enemy. Use target impact effects instead. |

## Clear Hit And Flash On Fade End

Enabled: particles are cleared at shutdown. For LineRenderer, Flash is cleared and disabled immediately when Hold ends or DisablePrepare() is called.

Disabled: emission stops, but existing Hit/Flash particles can finish naturally. With Deactivate When Finished enabled, automatic deactivation waits for these remaining particles.

**PARTICLE TAILS ARE NOT CONTINUOUS EMISSION**

Disabling this option preserves existing particles; it does not keep emitting forever. Use Continuous to keep the beam active. External SetActive(false) or Destroy still cuts off the effect.

## Two different kinds of hit effect

Hit Effect is a child of the laser and follows its blocking endpoint. Enemy Hit Effect Prefab creates separate effects on recognized targets through the optional pooling integration. These settings serve different purposes and can be used together.

06 / USER GUIDE

# Collision & damage

The main beam checks regular 3D colliders along its central axis. Trigger colliders are ignored. Visual width does not create a wide physical hit volume.

| | |
|---|---|
| Collision Mask | Layers included in ray, overlap and target-search queries. |
| Damage Over Time | Damage per second. Each emitting frame applies damageOverTime × Time.deltaTime. |
| Enemy Layers | Optional enemy classification for targets without the damage interface. |
| Pierce Enemies | Passes through recognized enemies. Other colliders still block the main beam. |

A target is recognized as an enemy when a component on the collider or a parent implements IHS_LaserDamageable, or the collider's layer is in Enemy Layers. That layer must also be included in Collision Mask.

## Connect your own health component

```
using UnityEngine;
using Hovl;
public class TargetHealth : MonoBehaviour, IHS_LaserDamageable
{
    public float health = 100f;
    public void TakeDamage(float damage)
    {
        health -= damage;
        if (health <= 0f) gameObject.SetActive(false);
    }
}
```

Alternatively, subscribe to On Hit. It provides the target GameObject and damage already calculated for the frame: do not multiply it by deltaTime again. It can also report blocking objects without the interface, so filter them if needed.

**AVOID APPLYING DAMAGE TWICE**

The interface and On Hit can both run for one hit. Use only one to apply the same health reduction. At zero Damage Over Time, On Hit is not invoked; visual impacts can still work.

07 / USER GUIDE

# Connect an existing enemy

Keep your existing movement, AI, health and death logic. HS_Laser only needs a way to deliver damage to that system. It does not require a specific Enemy class, an Enemy tag or a particular health implementation.

## Option A: add the interface directly

Add Hovl.IHS_LaserDamageable to your enemy component and implement public void TakeDamage(float damage). Inside that method, call your existing health reduction method. If you already have a matching public TakeDamage(float) method, adding the interface may be enough.

## Option B: use a small adapter

Use an adapter when you want to leave the existing enemy script unchanged. In this example, EnemyHealth is your existing component and ApplyDamage(float) is its damage method. Replace both names with the ones in your project.

```
using UnityEngine;
using Hovl;
[RequireComponent(typeof(EnemyHealth))]
public class LaserDamageAdapter : MonoBehaviour,
    IHS_LaserDamageable
{
    private EnemyHealth health;
    void Awake()
    {
        health = GetComponent<EnemyHealth>();
    }
    public void TakeDamage(float damage)
    {
        if (health != null) health.ApplyDamage(damage);
    }
}
```

Place the adapter beside the health component on the enemy root. Put its non-trigger 3D colliders on that root or its children: the laser searches upward from each collider for the interface. Use one receiver for the enemy's health.

**NO MANUAL REGISTRATION REQUIRED**

The interface automatically classifies the target as an enemy for piercing and chain selection. Include every collider layer in Collision Mask. Enemy Layers can remain empty with this approach.

08 / USER GUIDE

# Event-based enemy integration

Use On Hit when you prefer a central damage router. This approach needs no interface on the enemy. Add the following bridge to the laser object; replace EnemyHealth and ApplyDamage with your own API.

```
using UnityEngine;
using Hovl;
[RequireComponent(typeof(HS_Laser))]
public class LaserDamageBridge : MonoBehaviour
{
    private HS_Laser laser;
    void Awake() { laser = GetComponent<HS_Laser>(); }
    void OnEnable() { laser.onHit.AddListener(ApplyHit); }
    void OnDisable() { laser.onHit.RemoveListener(ApplyHit); }
    void ApplyHit(GameObject target, float damage)
    {
        if (!target) return;
        var health = target.GetComponentInParent<EnemyHealth>();
        if (health != null) health.ApplyDamage(damage);
    }
}
```

For this example, keep EnemyHealth on the collider object, its Rigidbody root or an ancestor. Without an interface, the event target is the attached Rigidbody GameObject when available; otherwise it is the collider GameObject.

| | |
|---|---|
| Enemy recognition | Put the enemy collider layers in both Enemy Layers and Collision Mask. This enables piercing, chains and target impact classification. |
| Compound colliders | Use a shared Rigidbody root for one enemy identity, or prefer the interface adapter. Otherwise separate collider objects can be treated as separate targets. |
| Damage amount | The float is already damage for this frame. Do not multiply by deltaTime or round each small hit to an integer. |
| Death and pooling | Your health system decides how death works. Disable or remove dead target colliders so they stop participating; reset health when reusing a pooled enemy. |

**CHOOSE ONE DAMAGE ROUTE**

Do not use this bridge and a damage-applying interface on the same target. Both can run for the same hit. The example EnemyHealth API is illustrative, not an additional required script supplied by HS_Laser.

09 / USER GUIDE

# Origin damage & chains

## Damage near the origin

Damage Near Origin adds a sphere at the beam origin, using Origin Damage Radius in world units. It shares Collision Mask and enemy recognition with the beam, and can detect a target whose collider contains the beam origin.

One laser damages each target at most once per frame across its main beam, origin sphere and chain. Different lasers apply damage independently. Damage runs during Growth and Hold, and stops when Fade starts.

> **ORIGIN SPHERE BEHAVIOR**
>
> The origin sphere does not perform a separate line-of-sight check against walls. Set its radius with this behavior in mind, or disable Damage Near Origin.

## Chain Bounces

Chains start from the first recognized enemy hit by the main beam and continue to nearby, unvisited targets. Hitting only with the origin sphere does not start a chain.

| | |
|---|---|
| Max Chain Bounces | Additional targets after the first hit: 0–100. |
| Chain Bounce Range | Search radius for the next target from the current link. |
| Chain Growth Duration | Extension time of each extra link. Zero makes transitions immediate. |

Targets need non-trigger 3D colliders within Collision Mask. Walls block links, and already visited targets are excluded. Chain Bounces works independently of Pierce Enemies: the main beam can continue while a chain starts from its first enemy.

Visual links are created when needed and reused by that laser instance. These are target-to-target transitions, not mirror reflections from walls.

10 / USER GUIDE

# Flicker & target impacts

## Width flicker

Enable Flicker Width to animate visual width between random values. Each instance starts with an independent random phase.

| | |
|---|---|
| Flicker Speed | Approximate change frequency. Zero freezes the current width. |
| Flicker Interval Variation | Random timing variation. At 0.5, intervals are approximately 50–150% of 1 / Flicker Speed. |
| Min / Max Width Multiplier | Lower and upper limits of the visual width multiplier. |

Flicker does not change beam length, damage or collision thickness.

## Pooled effects on targets

Enemy Hit Effect Prefab uses the optional HS_LaserHitEffects integration. Without that script in the project, HS_Laser still compiles, renders and deals damage; pooled target impacts are unavailable. Child Hit/Flash effects remain independent.

| | |
|---|---|
| Enemy Hit Effect Interval | Minimum interval on one target, shared by lasers using the same effect prefab. |
| Enemy Hit Effect Max Lifetime | Safety timeout for looping or other effects that have not finished. |
| Max Active Enemy Hit Effects | Shared active-effect budget per prefab. |
| Ensure Every Hit Target | Allows growth beyond the budget for new targets. Disable for a strict cap that may skip effects. |
| Follow Enemy Hit Target | Makes the effect follow its target. |
| Enemy Hit Effect Offset | Moves the effect away from the hit surface. |

11 / USER GUIDE

# Control from code

The component type is Hovl.HS_Laser. For explicit control, disable Play On Enable before the object's first activation.

```
// Activate and start a shot.
laser.gameObject.SetActive(true);
laser.Fire(playerTransform);
// Begin the configured shutdown.
laser.DisablePrepare();
```

The owner passed to Fire() and its child colliders are ignored by collision queries. Calling Fire() again restarts the shot and clears previous particles.

Message-based shutdown is also supported:

```
laser.gameObject.SendMessage(
    "DisablePrepare",
    SendMessageOptions.DontRequireReceiver);
```

| | |
|---|---|
| CurrentLength | Actual current length, including growth and blocking hits. |
| CurrentWidthMultiplier | Current visual flicker multiplier. |
| IsEmitting | True during an active shot before Fade. |
| IsBusy | True during the shot, including Fade. Does not include waiting for particle tails afterward. |
| TotalDuration | Current Growth + Hold + Fade settings; infinity for Continuous. Excludes Hit/Flash particle tails. |

**ADVANCED INTEGRATION**

If a hit callback changes a target's health components or hierarchy, call HS_Laser.InvalidateDamageReceiverCache() to clear cached receiver lookups.

12 / USER GUIDE

# Troubleshooting

## The laser is invisible

Check Laser Type, a renderer on the same GameObject, compatible material, +Z direction, nonzero Max Length and Fire / Play On Enable. ParticleMesh needs an appropriate mesh setup and shader.

## Laser Color does not change the line

Laser Color applies only to ParticleMesh. Use the LineRenderer gradient or its material color.

## Texture tiling does not respond

The script tiling controls apply only to LineRenderer. Check the noise Reference Name and whether the shader uses *_ST. For ParticleMesh, use material controls.

## Pierce Enemies does not work

Enable Pierce Enemies and ensure the target has the damage interface or belongs to Enemy Layers. An unrecognized collider is treated as an obstacle.

## The laser deals no damage

Check a non-trigger 3D collider, Collision Mask, positive Damage Over Time and either the interface or an On Hit handler. A material cannot apply damage by itself.

## The line disappears instead of fading

This is the current LineRenderer behavior. Shader-driven dissolve is implemented for ParticleMesh.

## Hit or Flash particles stop too early

Disable Clear Hit And Flash On Fade End. Ensure an external controller does not disable or destroy the object before the particles finish.

## Chain Bounces does not start

The main beam must hit a recognized enemy. Another target must be within Chain Bounce Range without an intervening wall. Use Max Chain Bounces above zero and Chain Growth Duration = 0 for brief contacts.